

Windowed Minority Guidance: Preliminary Evidence for Timestep-Localized Effects in Diffusion Denoising

Vittoria Lanzo — Independent Researcher

Abstract

Minority guidance (Um et al., 2024) steers diffusion sampling toward under-represented data regions via a classifier gradient at every denoising timestep. We test whether this effect is timestep-localized, comparing three equal-thirds windows — early ($t \in [0, 333]$), mid ($t \in [333, 667]$), late ($t \in [667, 1000]$) — against full-chain and no-guidance baselines over 50 seeds on LSUN Bedroom (250 runs). Mid recovers 0.456 of full-chain loss reduction; Wilcoxon tests confirm effects for mid, early, full vs. baseline (all $p < 0.001$), with late vs. baseline narrowly missing correction ($p = 0.0072$ vs. $\alpha/7 = 0.00714$). All windows produce detectable effects — preliminary evidence for mid-chain dominance without strict localization.

1. Introduction

Classifier-guided diffusion models (Dhariwal & Nichol, 2021) steer generation with classifier gradients at every denoising step. Um et al. (2024) extend this to *minority guidance*, directing sampling toward underrepresented regions. Because denoising timesteps differ functionally (Ho et al., 2020) and timestep-localized interventions produce qualitatively different effects (Meng et al., 2022), the mid-chain phase may be most receptive to minority gradient signals — enabling cheaper inference-time guidance if guidance there recovers most of the full-chain effect. We test this directly; Section 4 reports findings that partially challenge it.

2. Experiment

We use the LSUN Bedroom 256×256 model with the frozen minority classifier chain of Um et al. (2024) (ImageNet extractor + `mc_lsun.pt`), class 99, `guidance_scale=1.0`, `timestep_respacing=250` (Um et al. use 3.5; results may not generalize). Five conditions: *baseline*, *minority-full* ($t \in [0, 1000]$), *minority-early* ($t \in [0, 333]$), *minority-mid* ($t \in [333, 667]$), *minority-late* ($t \in [667, 1000]$). Guidance is zeroed outside the active window. We run 50 seeds × 5 conditions = 250 runs (Kaggle P100, float32) (*pre-registered n=10, expanded to 50; whether seeds 1–10 were observed first is unrecorded, making effective α uncertain*).

Primary metric: classifier cross-entropy loss at class 99 (lower = stronger guidance). Softmax confidence is corroboration only — distributions are severely right-skewed (mean/median: full 107×, late 154×, mid 46×, early 58×). (*Switching from pre-registered confidence to loss is an undisclosed protocol deviation; confidence-based effect sizes are an order of magnitude smaller.*)

3. Results

Table 1. Condition statistics (n=50; lower loss = stronger guidance).

Condition	Mean Loss	CV	Win Rate	Relative Effect
baseline	11.978	0.19	—	0.000
minority-early	11.129	0.23	50/50	0.162
minority-mid	9.591	0.35	39/50	0.456
minority-late	10.717	0.30	34/50	0.241
minority-full	6.741	0.61	44/50	1.000

% of Full Effect = $(\text{baseline} - \text{cond}) / (\text{baseline} - \text{full}) \times 100$; not a causal decomposition (windows interact across the trajectory). Win rate $\neq$ effect size.

The rank ordering **full > mid > late > early > baseline** holds across both loss and confidence. Table 2 reports Wilcoxon signed-rank tests on 50 paired seed-level loss differences. Mid, early, full vs. baseline are clearly significant ($p < 0.001$). Late vs. baseline ($p = 0.0072$) narrowly fails Bonferroni correction ($\alpha/7 = 0.00714$); mid vs. late ($p = 0.0166$) and late vs. early ($p = 0.39$) also do not survive correction.

Table 2. Wilcoxon signed-rank tests (n=50 pairs; Bonferroni threshold $\alpha/7 \approx 0.0071$).

Comparison	W	p-value	r _{rb}
mid vs. baseline	120	< 0.0001	0.812
late vs. baseline	362	0.0072	0.432
early vs. baseline	0	< 0.0001	1.000
full vs. baseline	48	< 0.0001	0.925
mid vs. late	391	0.0166	0.387
late vs. early	547	0.3882	0.142
full vs. mid	181	< 0.0001	0.716

$r_{rb} \in [0,1]$: rank-biserial correlation (effect size); $r_{rb} > 0$ indicates first condition has lower loss. Survives Bonferroni correction if $p < \alpha/7 \approx 0.0071$.

Mid recovers 0.456 of the full-chain loss reduction but sits closer to baseline than full-chain (2.39 vs. 2.85 units). Pre-registered criterion: mid mean loss ≤ 9.582 (baseline $\times 0.80$; distinct from the 0.456 share of full-chain effect recovered). Observed: 9.591 (missed by 0.009). Bootstrap 95% CI [8.69, 10.48] contains the threshold; criterion requires a point estimate. Early beats baseline in all 50 seeds ($r_{rb} = 1.000$), contradicting the “small or no effect” prediction for high-noise guidance. Late guidance occasionally worsens minority affinity: highest single-seed loss of any condition (17.04 vs. baseline 16.33), consistent with fine-detail guidance distorting features before global structure is established.

4. Discussion and Conclusion

Mid-window guidance recovers 0.456 of the full-chain effect — the largest single-window contribution but insufficient alone. All three windows produce detectable effects, contradicting binary localization. **Key finding:** early-window guidance achieves perfect seed-wise consistency ($r_{rb}=1.000$, 50/50 wins) despite the assumption that high-noise gradients are uninformative — evidence that minority signals propagate throughout the denoising trajectory. **Revised framing:** mid-chain is quantitatively dominant but the effect is distributed; no single window approximates full-chain guidance. Late > early is descriptive only ($p = 0.39$).

Limitations: single configuration (LSUN Bedroom, class 99, guidance_scale=1.0); n=1 per seed; high full-chain variance (CV=0.61); no perceptual metrics. Mid-window guidance may offer cheaper inference-time minority guidance, but requires validation at the published guidance_scale=3.5.

References

- Dhariwal & Nichol (2021). Diffusion Models Beat GANs. *NeurIPS 2021*.
- Ho, Jain & Abbeel (2020). Denoising Diffusion Probabilistic Models. *NeurIPS 2020*.
- Meng et al. (2022). SDEdit: Guided Image Synthesis via SDEs. *ICLR 2022*.
- Um, Lee & Ye (2024). Don’t Play Favorites: Minority Guidance for Diffusion Models. *ICLR 2024*.

Appendix — Reproducibility

A.1 Experimental Configuration

Models and checkpoints (all publicly available):

Component	Checkpoint	Source
Diffusion model	lsun_bedroom.pt	OpenAI guided-diffusion
Feature extractor	256x256_classifier.pt	OpenAI guided-diffusion
Minority classifier	mc_lsun.pt	Um et al. (2024)

Sampler hyperparameters (all fixed across conditions):

Parameter	Value
diffusion_steps	1000

timestep_respacing	250
noise_schedule	linear
guidance_scale	1.0
image_size	256
num_channels	256
num_res_blocks	2
attention_resolutions	32,16,8
learn_sigma	True
use_fp16	False
manual_class_id	99
num_samples	1

The minority classifier (`mc_lsun.pt`) expects 8×8 latent features produced by `256x256_classifier.pt` with `f_out=True` (see A.2). Its config: `image_size=8`, `in_channels=512`, `out_channels=100`, `classifier_width=128`, `classifier_pool=attention`.

UNET.PY PATCH — the upstream repository’s `EncoderUNetModel.forward` does not expose intermediate features. The following minimal patch is required:

```
# guided_diffusion/unet.py - add f_out parameter to EncoderUNetModel.forward
def forward(self, x, timesteps, f_out=False):
    ...
    # (existing body unchanged until final return)
    if f_out:
        return h          # 8x8 intermediate feature map (B, 512, 8, 8)
    return self.out(h)    # original logits
```

Data schema (`data/experiment_runs.csv`, 250 rows, one per run):

```
run_id | condition | guidance_window | t_start | t_end | guidance_scale | seed | n_generated |
classifier_mean_confidence | classifier_mean_loss | runtime_seconds | artifact_path |
timestamp
```

Metrics are computed at $t=0$ (clean image) by passing generated images through the frozen classifier chain. Each run produces one image; `classifier_mean_*` fields are single-sample values ($n=1$ per seed per condition).

A.2 Windowed Guidance Implementation

The windowed guidance requires no architectural modification — it is implemented entirely within the `cond_fn` callback:

```
t_start = args.t_start # e.g. 333 for mid window
t_end   = args.t_end   # e.g. 667 for mid window

def cond_fn(x, t, y=None):
    current_t = int(t[0].item())
    # Zero gradient outside the active window
    if not (t_start <= current_t < t_end):
        return torch.zeros_like(x)
    with torch.enable_grad():
        x_in = x.detach().requires_grad_(True)
        latents = f_extractor(x_in, timesteps=t, f_out=True) # → (B, 512, 8, 8)
        logits = classifier(latents, timesteps=t)
        log_probs = F.log_softmax(logits, dim=-1)
        selected = log_probs[range(len(logits)), y.view(-1)]
        # Scale applied here; sampler must NOT re-apply classifier_scale
        # (differs from default OpenAI codebase convention).
        return torch.autograd.grad(selected.sum(), x_in)[0] * args.classifier_scale
```

For baseline runs, `guidance_scale=0.0` is passed rather than gating by timestep. Window boundaries use raw diffusion timesteps ($t=1000$: pure noise $\rightarrow t=0$: clean image); the active window for mid is $t \in [333, 667]$, meaning guidance fires during 334 of the 1000 raw timesteps (≈ 134 of the 250 respaced steps).

A.3 Statistical Analysis Code

```
import csv, random
from scipy import stats

# --- Load paired losses ---
data = {}
with open('data/experiment_runs.csv') as f:
    for row in csv.DictReader(f):
        data.setdefault(row['condition'], {})[int(row['seed'])] = \
            float(row['classifier_mean_loss'])

seeds = sorted(data['baseline'])
get = lambda cond: [data[cond][s] for s in seeds]
baseline, early, mid, late, full = (get(c) for c in
    ['baseline', 'minority-early', 'minority-mid', 'minority-late', 'minority-full'])

# --- Wilcoxon signed-rank tests ---
def wilcoxon_rbc(a, b):
    diffs = [ai - bi for ai, bi in zip(a, b)]
    # zero_method='zsplit': split zero-difference ranks equally across W+ and W-,
    # keeping denominator = n(n+1)/2 = 1275 valid for all 7 tests.
    W, p = stats.wilcoxon(diffs, zero_method='zsplit')
    r_rb = 1 - (2 * W) / (len(diffs) * (len(diffs) + 1) / 2)
    return W, p, r_rb

comparisons = [
    (mid, baseline, 'mid vs. baseline'),
    (late, baseline, 'late vs. baseline'),
    (early, baseline, 'early vs. baseline'),
    (full, baseline, 'full vs. baseline'),
    (mid, late, 'mid vs. late'),
    (late, early, 'late vs. early'),
    (full, mid, 'full vs. mid'),
]
for a, b, label in comparisons:
    W, p, r = wilcoxon_rbc(a, b)
    print(f'{label:25s} W={W:6.0f} p={p:.4f} r_rb={r:.3f}')

# Bonferroni-corrected threshold for 7 simultaneous tests
alpha_corrected = 0.05 / 7 # = 0.00714

# --- Bootstrap 95% CI for mid mean loss (10,000 resamples) ---
random.seed(42)
boot = sorted(sum(random.choices(mid, k=50)) / 50 for _ in range(10_000))
ci = (boot[249], boot[9749]) # 2.5th and 97.5th percentiles → [8.69, 10.48]

# Pre-registered threshold: mid mean loss ≤ baseline_mean × 0.80
baseline_mean = sum(baseline) / len(baseline) # 11.978
threshold = baseline_mean * 0.80 # 9.582
mid_mean = sum(mid) / len(mid) # 9.591 (misses by 0.009)
```

Environment: Python 3.11, scipy 1.x, numpy 2.x. All 250 runs generated on Kaggle (NVIDIA Tesla P100, float32, CUDA 11.x). The full Kaggle notebook (including model download, patching, sampling loop, and CSV export) is available on request.